

In this journal entry, I will share my thoughts about module 5 and fixing datasets for machine learning. After learning how to do the preprocessing of the machine learning dataset, my main takeaway is how to deal with different situations that may occur when it comes to the preprocessing of machine learning datasets. Using the same Titanic dataset from module 4, it helped me see more problems with raw data and how some of these may happen in real time.

Before the lab started, I thought that data preparation was just about removing anything incomplete. What I didn't expect was the amount of estimation and educated guessing while doing the preparation. When I was working with the age and Cabin columns, it showed that more than half of them were empty. This made me think that the best solution to fix the problem is just to delete them. But the lab addressed this as not the best solution and rather wanted me to impute the data with values that were guessed. At first, this came as a surprise to me, as to me, this could throw off the AI because if the information is guessed, then it may lead to bad AI. However, when I worked through task 1, I realized that assuming is better than nothing. As if you want to delete that row and other information that may be useful for later, like the fare, is gone as well. I also realized that the guesses were based on the dataset instead of outside information. As the median may not be exactly the age of the passenger, it will be better than losing the other parts of the data.

This realization came during task one, when I was asked to fill in the Age column and was told to use the median instead of the mean. At first, I questioned the decision, as the mean is used for most averages of values in a list and is quite accurate. However, as I wrote code for both the median and the mean, I realized that the median is much better at making sure outliers don't affect the data. Elderly passengers can pull the mean up, while the median is the true middle. In this dataset, the difference was only 1.7, which shows that even though it may not make a huge difference, there still is one.

After working with missing values, the next problem I faced was that instead of values not being there, they were in the wrong format. In the sex column. If we gave this information to an AI, it could mislead it in the wrong direction. The first thought in mind was to just give each category a number. However, the lab showed the one-hot encoding method. This method, I noticed, was like a binary system to display the categories, where 1 means that it's true and 0 means that it's not true. When looking over the code, I noticed that we were deleting a column,

but after thinking about it makes logical sense as if Sex\_Male is false, then we know they are a female. However, I still wanted to know why just assigning a number to each category wasn't a good solution, as this method is called Ordinal Encoding, where each category gets an ascending order number. However, after I investigated it more, I realized that this almost always throws the AI off, as the AI will take the higher number as being better than the other, but it could just be different colors, and one is not better than the other.

Feature scaling is something I have never heard before, but I now realize that it's great for making sure the AI isn't dominated by one feature and making sure everything is equal. I saw the age and fare go from 22 and 71 down to -0.56 and 0.78. I thought the data was being broken down and was less important, but I now understand that the standard scaler was taking the age, subtracting it from the mean, and then dividing by the standard deviation, so I now know that this is not a mistake and the passenger was just below average. The reason for these matters as fare can go up 512 while the age only goes up to 80. If you keep them like they are, the AI will pay more attention to the much higher number.

Module 5 showed me that even though making the model may be hard, the prep is much more time consuming than any other. Every step you take to make the data for the machine is a tedious process that requires careful consideration, as this part can affect the rest of your time building the model, and the better your data preparation, the better your machine can become.

Monepha Mitchell

Professor Viswanatha Rao (Vishwa)

ITAI 1371 Introduction to Machine Learning

27 Jun 2026

### Reflective Journal

This lab helped me better understand why data preparation is such an important step before building a machine learning model. I learned that even if a dataset contains useful information, it may have missing values, categorical data, or features with different scales that need to be processed first. Completing each step helped me understand how these preprocessing techniques improve data quality and prepare it for machine learning.

One of the first things I worked on was filling in the missing values in the Age column using the median. I learned that the median is often a better choice than the mean because it is less affected by extreme values. After that, I used One-Hot Encoding to convert the categorical columns into numerical values that a machine learning model can understand. Finally, I applied feature scaling to the Age and Fare columns using StandardScaler. This showed me how scaling helps put numerical features on a similar range, which is important for many machine learning algorithms.

I also enjoyed working with my group during this lab. Although we each completed our own notebooks, we communicated throughout the assignment to discuss any challenges we encountered and to help each other understand the preprocessing steps. Sharing ideas made it easier to complete the lab and ensured that everyone understood the concepts before submitting their work.

Overall, this lab gave me more confidence in using Python to prepare data for machine learning. It reinforced that data cleaning and preprocessing are essential parts of the machine

learning process. I know these skills will be useful in future labs and in our upcoming midterm and final projects, where correctly preparing a dataset will be just as important as building the machine learning model itself.

## Reflective Journal: Module 05 Lab - Data Preparation

Module 05 helped me understand that machine learning is not just about choosing a model and running it. A big part of the process happens before the model ever gets trained. In Module 04, we focused more on exploring the data and finding patterns, but this lab moved into the next step: actually preparing the data so a machine learning model can use it. That made the whole workflow feel more realistic to me because raw data is usually messy, incomplete, and not automatically ready for modeling.

The Titanic dataset was a good example for this lab because it had common problems that show up in real datasets. One of the first issues was missing values, especially in the Age column. Instead of just deleting rows with missing ages, we used median imputation to fill them in. I thought this made sense because deleting too many rows could remove useful information, and the median is safer than the mean when there are outliers or skewed values. This helped me see that cleaning data is not just a technical step. It also involves making choices and understanding why one method may be better than another.

Another important part of the lab was encoding categorical data. Before this module, I understood that models need numbers, but this lab made that idea more practical. Columns like Sex and Embarked are meaningful to humans, but a machine learning model cannot directly understand words like "male," "female," or "S." One-hot encoding fixed that by turning categories into separate numeric columns with 0s and 1s. This was one of the clearer examples of data preparation for me because it showed how we translate real-world information into something a model can actually process.

The scaling part of the lab also helped me understand why feature size matters. Age and Fare are both numerical columns, but they exist on different scales. A fare value can be much larger than an age value, and certain models might accidentally treat the larger numbers as more important just because of their size. Using StandardScaler made the values more comparable by centering them around a common scale. I also liked learning that scaling is not equally important for every model. For example, Decision Trees usually do not need scaling because they split data based on conditions rather than distance.

The key readings and course presentation also helped me connect this lab to the bigger picture of feature engineering. One idea that stood out to me is that feature engineering is about improving how the data represents the actual problem. Good features can help a simple model perform better, while on the other hand messy or poorly chosen features can hurt even a strong model. That made me think differently about machine

learning because the “smart” part is not only the algorithm. A lot of the intelligence comes from how carefully the data is prepared.

One challenge in this lab was making sure the notebook structure and code were clean before running everything. Like the previous lab, some of the code needed to be placed carefully so each command ran on its own line. This reminded me that small formatting or syntax issues can stop the whole notebook, even when the overall logic is correct. I am starting to get more comfortable debugging those issues, but it still takes patience.

Overall, this lab made data preparation feel more important and less like a background step. I learned how missing values, categorical variables, and scaling can all affect whether a dataset is usable for machine learning. The biggest takeaway for me is that a model can only learn from the data it is given. If the data is incomplete, messy, or in the wrong format, the model’s results will probably suffer. Going forward, I think I will pay more attention to the preparation stage before jumping into model training because this lab showed me that clean, thoughtful data is one of the foundations of good machine learning.